BareMinimum

Real-world use and code guide
Personal wellness baseline tracker | Updated November 2026

> **Purpose:** BareMinimum helps an individual record sleep and weekly activity, compare self-reported numbers with public health guidance, and review changes over time. It is a wellness tracking tool, not a medical device, diagnosis, or treatment service.

1. What is grounded in real-world guidance?

The health panel records four manual inputs: last night's sleep hours, moderate activity minutes this week, vigorous activity minutes this week, and strength-training days this week. It labels them as self-reported, records the save time, and shows official guidance links.

- **Sleep:** CDC guidance says adults aged 18-60 generally need at least 7 hours; recommendations vary by age. One night is a snapshot and does not establish a sleep disorder.
- **Aerobic activity:** WHO guidance for adults is 150-300 minutes of moderate-intensity activity weekly, or an equivalent amount of vigorous activity. The dashboard estimates moderate-equivalent minutes as moderate + 2 x vigorous.
- **Strength:** WHO and CDC guidance recommends muscle-strengthening activity on at least 2 days each week for adults.

Sources to verify:
CDC - About Sleep
WHO - Physical activity
CDC - Adult Activity: An Overview
WHO - Guidelines on physical activity and sedentary behaviour

2. Practical way to use it

1. Open the SDG synergy details and enter sleep and activity values from your own notes or tracker.
2. Choose minutes for the current week; record minutes rather than estimating an app score.

3. Save. If signed in and MongoDB is configured, the record is saved to your private account. Otherwise it stays in this browser's local storage.
4. Review the displayed comparison with the linked guidance. Use the source links to check population, age, and details that apply to you.
5. Repeat regularly to build a personal history. The current version saves the latest baseline; it is not yet a dated longitudinal journal or wearable integration.

3. Data and database design

MongoDB uses two collections. **dashboardSnapshots** stores the user's dashboard snapshot. **healthRecords** stores the latest health baseline. Both use the authenticated OAuth `openId` as the document key. The API gets that ID from the verified server session; the browser cannot choose another user's ID. Input ranges are validated on the server.

```
healthRecords document
_id: authenticated user's openId
sleepHours: number (0-24)
moderateMinutes: number (0-1000)
vigorousMinutes: number (0-1000)
strengthDays: number (0-7)
recordedAt: ISO timestamp
source: "self_report"
references: official guidance URLs
```

If a user is not authenticated, the app does not write health values to shared MongoDB. It saves them in browser localStorage and displays a local-only status. Configure OAuth and MongoDB to enable private cross-device storage.

4. Code structure and request flow

```
client/src/pages/Home.tsx
  Main dashboard, layout, and dashboard snapshot sync
client/src/app/components/DashboardActionDialog.tsx
  Opens the SDG panel and embeds the health tracker
client/src/app/components/HealthTracker.tsx
  Health inputs, guidance, local cache, save/load requests
client/src/app/components/SdgSynergy.tsx
  Metric cards and honest sample-data labels
client/src/app/models/dashboard.ts
  TypeScript model and sample dashboard values
server/_core/index.ts
  Express routes, auth checks, validation, server startup
server/_core/sdk.ts
  Verifies the app session and resolves the authenticated
user
server/mongo.ts
```

```
Mongo connection plus per-user collection reads/writes
```

1. React renders the dashboard and health form.
2. The form stores an offline copy in localStorage, then sends a request to `/api/health-record`.
3. Express verifies the session using `sdk.authenticateRequest`. Unauthenticated requests receive HTTP 401.
4. The server validates allowed ranges and passes the verified user ID and record to `server/mongo.ts`.
5. MongoDB upserts that user's document. A load request retrieves only the matching user's record.

5. Libraries and why they are used

- **React:** component-based interactive interface and local state.
- **TypeScript:** catches mismatched data shapes between UI and API at build time.
- **Vite:** local development server and frontend production build.
- **Express:** HTTP endpoints for dashboard and health data.
- **MongoDB Node driver:** persists documents in collections.
- **Lucide React:** consistent interface icons.
- **Radix UI Dialog:** accessible modal behavior for the details panel.
- **localStorage:** browser-only fallback when login or database sync is unavailable.

6. What is real and what is still sample data?

The sleep and activity values entered in the health panel are real inputs supplied by the user. The app does not measure them automatically. Existing work-focus, carbon, profile, and activity-feed values remain illustrative sample data and are labeled as such. The Oura item is not connected. No wearable, calendar, or carbon-accounting source is currently integrated.

For a production deployment, enable authenticated login, configure MongoDB, add a dated observation history and export/delete controls, add consent and retention rules, and conduct privacy and accessibility review. Avoid entering sensitive medical history. Secure secrets in server environment variables, never in client code.

7. Start locally

In PowerShell from the project directory, set `PORT=3001` and run `pnpm dev`. Visit `http://127.0.0.1:3001`. The OAuth environment must include `MANUS_OAUTH_API_URL` for sign-in, and Mongo sync requires `MONGODB_URI` and optionally `MONGODB_DB_NAME`. Without sign-in, local-only entry remains available.

This guide documents the current implementation. Health references are informational and do not replace advice from a qualified health professional.